

Formalisation du jeu Songho / Songo et algorithme complet

1. 1. Objectif du document

Ce document sert de reference technique pour programmer le jeu Songho/Songo dans deux versions :

- Version 1 : deux joueurs jouent sur le meme navigateur, sur le meme site.
- Version 2 : deux joueurs jouent depuis deux sites/navigateurs distants avec Ajax.

Le document fixe :

- les regles retenues ;
- les decisions prises apres recherche sur les points ambigus ;
- le modele de donnees ;
- les algorithmes complets ;
- les cas limites ;
- les fonctions a programmer.

Il fixe aussi la separation entre les deux modes de deploiement :

Version	Mode de jeu	Hebergement recommande
Version 1 locale	Deux joueurs sur le meme ecran et le meme navigateur.	Cloudflare Pages
Version 2 distante	Deux joueurs sur deux navigateurs relies par Ajax.	Render

Cette decision permet de garder la version 1 simple et statique, puis de reserver le serveur a la version 2.

2. 2. Sources consultees

Sources principales :

- Club Awale, "Le Songo : regles du jeu" : <https://www.clubawale.com/post/comment-jouer-le-songo>
- Regles du Songo sur Escale a Jeux, PDF : https://regle.escaleajeux.fr/songo_rg.pdf
- Article "Les regles du jeu de Songo" : <https://lasdebrouille.blogspot.com/2019/10/les-regles-du-jeu-de-songo.html>
- Page BoardGameGeek "Songo Ewondo" : <https://boardgamegeek.com/boardgame/137560/songo-ewondo>

Les sources confirment que le Songo a plusieurs variantes. Pour ce projet, on retient la variante camerounaise de type Ewondo/Bulu, car c'est celle decrite dans la source fournie.

3. 3. Decisions prises sur les ambiguetes

Point ambigu	Decision pour le projet	Justification
Orthographe	Le code peut utiliser <code>songo</code> , l'interface peut afficher <code>Songho</code> ou <code>Songo</code> .	Les sources utilisent surtout "Songo", mais le projet enseignant parle de "Songho".
Nombre de cases	7 cases par joueur, donc 14 cases.	Confirme par les sources.
Graines initiales	5 graines par case, total 70.	Confirme par les sources.
Seuil de victoire	40 graines capturees.	Confirme par Club Awale et les regles detaillees.
Case adverse protegee	La premiere case adverse atteinte par la semaille ne peut pas etre capturee seule par capture normale.	Les regles detaillees disent que la capture isolee dans cette case est interdite.
Capture de cette case dans une chaine	Autorisee si la chaine commence depuis une autre case adverse.	Confirme par Club Awale.
Coup depuis la case d'attaque avec 1 graine	Interdit en situation normale.	Les regles detaillees interdisent de donner une seule graine dans le camp adverse depuis la case d'attaque.
Coup depuis la case d'attaque avec 2 graines	Interdit, sauf si le coup provoque une capture.	Les regles detaillees signalent cette exception.
Cas force avec 1 ou 2 graines depuis la case d'attaque	Dans notre projet : les graines sont transferees au score de l'adversaire, sans semaille normale.	Club Awale dit que si le joueur est contraint, ces graines "reviennent a l'adversaire". Pour une implementation deterministe, on formalise cela comme un don au score adverse.
Grenier, c'est-a-dire case contenant plus de 13 graines	Faire un tour complet sans reposer dans la case source, puis distribuer les graines restantes seulement dans le camp adverse.	Confirme par les regles detaillees.
Derniere graine d'un grenier qui devrait tomber dans la premiere case adverse	Cette graine est capturee seule par le joueur actif et n'est pas ajoutee durablement au tablier.	Les sources donnent les exemples 14, 21, 28, 35 graines.
Capture qui viderait tout le camp adverse	Capture annulee entierement. La semaille reste valide.	Regle generale d'interdiction d'affamer l'adversaire.

Point ambigu	Decision pour le projet	Justification
Solidarite	Si le camp adverse est vide, il faut le nourrir avec au moins 7 graines si possible ; sinon il faut donner le maximum possible ; si rien n'est possible, la partie s'arrete.	Confirme par Club Awale et coherent avec les regles de semailles africaines.
Fin avec moins de 10 graines sur le tablier	La partie s'arrete, les graines restantes sont attribuees au proprietaire de leur camp.	Regle donnee par Club Awale.

4. 4. Vocabulaire

Terme	Definition
Tablier	Ensemble des 14 cases de jeu.
Camp	Ensemble des 7 cases d'un joueur.
Graine	Unite placee dans une case ou capturee dans un score.
Score	Nombre de graines capturees par un joueur.
Semer	Distribuer les graines d'une case une par une.
Capture	Retirer des graines du camp adverse et les ajouter au score du joueur actif.
Prise a la chaine	Capturer plusieurs cases adverses consecutives en remontant depuis la derniere case semee.
Solidarite	Obligation de donner des graines au joueur dont le camp est vide.
Grenier	Case contenant plus de 13 graines.
Case d'attaque	Derniere case de son camp avant d'entrer dans le camp adverse.
Premiere case adverse	Premiere case de l'adversaire atteinte apres la case d'attaque. Cette case est protegee contre la capture isolee.

5. 5. Coordonnees internes conseillees

Pour eviter les confusions entre l'affichage et le moteur, le moteur utilise des coordonnees physiques.

Affichage physique du tablier :

Nord : N0 N1 N2 N3 N4 N5 N6
Sud : S0 S1 S2 S3 S4 S5 S6

Correspondance avec les numeros humains de Club Awale :

```
Nord humain : 1  2  3  4  5  6  7
Nord code   : N0 N1 N2 N3 N4 N5 N6

Sud humain  : 7  6  5  4  3  2  1
Sud code    : S0 S1 S2 S3 S4 S5 S6
```

Ainsi :

- la case d'attaque de Nord est N6 ;
- la premiere case adverse pour Nord est S6 ;
- la case d'attaque de Sud est S0 ;
- la premiere case adverse pour Sud est N0 .

Cycle de semaille :

```
N0 -> N1 -> N2 -> N3 -> N4 -> N5 -> N6
-> S6 -> S5 -> S4 -> S3 -> S2 -> S1 -> S0
-> N0 ...
```

Dans ce cycle :

- un joueur Nord avance vers N6 , puis entre chez Sud par S6 ;
- un joueur Sud avance vers S0 , puis entre chez Nord par N0 .

6. 6. Constantes du jeu

```
const RULES = {
  players: ["north", "south"],
  pitsPerPlayer: 7,
  initialSeedsPerPit: 5,
  totalSeeds: 70,
  victoryScore: 40,
  lowBoardLimit: 10,
  captureValues: [2, 3, 4],
  maxNormalSowSeeds: 13
};
```

7. 7. Modele de donnees

Modele minimal :

```
const initialState = {
  board: {
    north: [5, 5, 5, 5, 5, 5, 5],
    south: [5, 5, 5, 5, 5, 5, 5]
  },
  scores: {
    north: 0,
    south: 0
  },
  currentPlayer: "south",
  status: "playing",
  winner: null,
  reason: null,
  moveNumber: 0,
  history: []
};
```

Types logiques :

```
// player = "north" ou "south"
// pitIndex = entier entre 0 et 6

const move = {
  player: "south",
  pitIndex: 2
};
```

Invariant obligatoire :

$$\text{score_north} + \text{score_south} + \text{somme_cases_north} + \text{somme_cases_south} = 70$$

Cet invariant doit etre verifie apres chaque coup pendant les tests.

8. 8. Fonctions utilitaires indispensables

```

function other(player) {
  return player === "north" ? "south" : "north";
}

function sum(values) {
  return values.reduce((total, value) => total + value, 0);
}

function boardSeeds(state) {
  return sum(state.board.north) + sum(state.board.south);
}

function totalSeeds(state) {
  return state.scores.north + state.scores.south + boardSeeds(state);
}

function isOpponentPit(player, position) {
  return position.player === other(player);
}

function attackPit(player) {
  return player === "north"
    ? { player: "north", pitIndex: 6 }
    : { player: "south", pitIndex: 0 };
}

function opponentFirstPit(player) {
  return player === "north"
    ? { player: "south", pitIndex: 6 }
    : { player: "north", pitIndex: 0 };
}

```

Cycle interne :

```

const CYCLE = [
  { player: "north", pitIndex: 0 },
  { player: "north", pitIndex: 1 },
  { player: "north", pitIndex: 2 },
  { player: "north", pitIndex: 3 },
  { player: "north", pitIndex: 4 },
  { player: "north", pitIndex: 5 },
  { player: "north", pitIndex: 6 },
  { player: "south", pitIndex: 6 },
  { player: "south", pitIndex: 5 },
  { player: "south", pitIndex: 4 },
  { player: "south", pitIndex: 3 },
  { player: "south", pitIndex: 2 },
  { player: "south", pitIndex: 1 },
  { player: "south", pitIndex: 0 }
];

```

9. 9. Algorithme de creation d'une partie

Entrees :

- joueur qui commence : "north" ou "south".

Sortie :

- etat initial.

Pseudo-code :

```
function createGame(startingPlayer = "south") {
  return {
    board: {
      north: [5, 5, 5, 5, 5, 5, 5],
      south: [5, 5, 5, 5, 5, 5, 5]
    },
    scores: { north: 0, south: 0 },
    currentPlayer: startingPlayer,
    status: "playing",
    winner: null,
    reason: null,
    moveNumber: 0,
    history: []
  };
}
```

10. 10. Generation du chemin de semaille

Fonction `nextPositionsAfter(source)` :

```
function samePosition(a, b) {
  return a.player === b.player && a.pitIndex === b.pitIndex;
}

function cycleIndexOf(position) {
  return CYCLE.findIndex((item) => samePosition(item, position));
}

function nextPositionsAfter(source) {
  const start = cycleIndexOf(source);
  const positions = [];

  for (let step = 1; step <= 13; step++) {
    const index = (start + step) % CYCLE.length;
    positions.push(CYCLE[index]);
  }

  return positions;
}
```

Cette fonction retourne les 13 autres cases dans l'ordre de semaille, sans jamais remettre une graine dans la case source pendant un tour complet.

11. 11. Algorithme de semaille normale

Condition :

```
1 <= graines_de_la_case_source <= 13
```

Pseudo-code :

```
function sowNormal(state, player, pitIndex) {
  const seeds = state.board[player][pitIndex];
  const source = { player, pitIndex };
  const visited = [];

  state.board[player][pitIndex] = 0;

  const path = nextPositionsAfter(source);

  for (let i = 0; i < seeds; i++) {
    const position = path[i];
    state.board[position.player][position.pitIndex] += 1;
    visited.push(position);
  }

  return {
    visited,
    lastPosition: visited[visited.length - 1],
    specialCapture: 0
  };
}
```

Exemple :

```
Sud joue S2 avec 5 graines.
Chemin : S1, S0, N0, N1, N2.
S2 devient 0.
Les cinq cases du chemin gagnent 1 graine.
```

12. 12. Algorithme de semaille d'un grenier

Condition :

```
graines_de_la_case_source > 13
```


Regle :

1. Le joueur vide la case source.
2. Il fait un tour complet en déposant une graine dans chacune des 13 autres cases.
3. Les graines restantes sont distribuées uniquement dans le camp adverse.
4. Si la dernière graine restante devrait tomber dans la première case adverse, cette dernière graine est capturée par le joueur actif et n'est pas ajoutée durablement au tablier.

Chemin adverse pour le reste du grenier :

```
function opponentPath(player) {  
  return player === "north"  
    ? [  
      { player: "south", pitIndex: 6 },  
      { player: "south", pitIndex: 5 },  
      { player: "south", pitIndex: 4 },  
      { player: "south", pitIndex: 3 },  
      { player: "south", pitIndex: 2 },  
      { player: "south", pitIndex: 1 },  
      { player: "south", pitIndex: 0 }  
    ]  
    : [  
      { player: "north", pitIndex: 0 },  
      { player: "north", pitIndex: 1 },  
      { player: "north", pitIndex: 2 },  
      { player: "north", pitIndex: 3 },  
      { player: "north", pitIndex: 4 },  
      { player: "north", pitIndex: 5 },  
      { player: "north", pitIndex: 6 }  
    ];  
}
```

Pseudo-code :

```

function sowGranary(state, player, pitIndex) {
  const seeds = state.board[player][pitIndex];
  const source = { player, pitIndex };
  const visited = [];
  let remaining = seeds;
  let specialCapture = 0;

  state.board[player][pitIndex] = 0;

  // 1. Tour complet sans la case source.
  for (const position of nextPositionsAfter(source)) {
    state.board[position.player][position.pitIndex] += 1;
    visited.push(position);
    remaining -= 1;
  }

  // 2. Reste uniquement chez l'adversaire.
  const path = opponentPath(player);

  for (let i = 0; i < remaining; i++) {
    const position = path[i % path.length];
    const isLastSeed = i === remaining - 1;
    const isProtectedFirstPit = samePosition(position, opponentFirstPit(player));

    if (isLastSeed && isProtectedFirstPit) {
      specialCapture += 1;
      visited.push(position);
      continue;
    }

    state.board[position.player][position.pitIndex] += 1;
    visited.push(position);
  }

  return {
    visited,
    lastPosition: visited[visited.length - 1],
    specialCapture
  };
}

```

Exemples :

Source = 14 graines
13 graines sont semees dans les 13 autres cases.
La 14e devrait tomber dans la premiere case adverse.
Elle est capturee seule par le joueur actif.

Source = 15 graines
13 graines sont semees dans les 13 autres cases.
La 14e est semee dans la premiere case adverse.
La 15e est semee dans la deuxieme case adverse.
Pas de capture speciale.

Source = 21 graines
13 graines sont semees dans les 13 autres cases.
7 graines sont semees dans les 7 cases adverses.
La derniere graine restante revient sur la premiere case adverse.
Cette derniere graine est capturee seule.

13. 13. Algorithme general de semaille

```
function sow(state, player, pitIndex) {  
  const seeds = state.board[player][pitIndex];  
  
  if (seeds <= 0) {  
    throw new Error("La case choisie est vide.");  
  }  
  
  if (seeds <= 13) {  
    return sowNormal(state, player, pitIndex);  
  }  
  
  return sowGranary(state, player, pitIndex);  
}
```

14. 14. Capture normale

Conditions :

1. La derniere graine est dans le camp adverse.
2. La derniere graine n'est pas dans la premiere case adverse protegee.
3. La case finale contient 2, 3 ou 4 graines apres la semaille.

Si les trois conditions sont vraies, la case finale est capturable.

```

function isCaptureValue(seedCount) {
  return seedCount === 2 || seedCount === 3 || seedCount === 4;
}

function canStartCapture(state, player, lastPosition) {
  if (!isOpponentPit(player, lastPosition)) return false;
  if (samePosition(lastPosition, opponentFirstPit(player))) return false;

  const count = state.board[lastPosition.player][lastPosition.pitIndex];
  return isCaptureValue(count);
}

```

15. 15. Prise a la chaine

La prise a la chaine se fait dans le camp adverse en revenant vers la premiere case adverse.

Exemple pour Nord :

Chemin adverse Nord : S6, S5, S4, S3, S2, S1, S0
 Si la derniere graine tombe en S3, on teste S3, puis S4, puis S5, puis S6.

Exemple pour Sud :

Chemin adverse Sud : N0, N1, N2, N3, N4, N5, N6
 Si la derniere graine tombe en N4, on teste N4, puis N3, puis N2, puis N1, puis N0.

Pseudo-code :

```

function captureChainPositions(state, player, lastPosition) {
  const path = opponentPath(player);
  const lastIndex = path.findIndex((position) => samePosition(position, lastPosition));

  if (lastIndex <= 0) {
    return [];
  }

  const captured = [];

  for (let index = lastIndex; index >= 0; index--) {
    const position = path[index];
    const count = state.board[position.player][position.pitIndex];

    if (!isCaptureValue(count)) {
      break;
    }

    captured.push({
      player: position.player,
      pitIndex: position.pitIndex,
      seeds: count
    });
  }

  return captured;
}

```

Important :

- si la chaine commence dans la premiere case adverse, elle est interdite ;
- si la chaine commence ailleurs et atteint la premiere case adverse, cette premiere case peut etre capturee ;
- la chaine s'arrete au premier trou qui ne contient pas 2, 3 ou 4 graines.

16. 16. Interdiction d'affamer l'adversaire

Une capture ne doit jamais vider completement le camp adverse.

Algorithme :

```

function wouldEmptyOpponent(state, player, captureList) {
  const opponent = other(player);
  const remaining = [...state.board[opponent]];

  for (const capture of captureList) {
    remaining[capture.pitIndex] -= capture.seeds;
  }

  return sum(remaining) === 0;
}

function applyCaptureIfAllowed(state, player, captureList) {
  if (captureList.length === 0) return 0;

  if (wouldEmptyOpponent(state, player, captureList)) {
    return 0;
  }

  let total = 0;

  for (const capture of captureList) {
    state.board[capture.player][capture.pitIndex] -= capture.seeds;
    total += capture.seeds;
  }

  state.scores[player] += total;
  return total;
}

```

17. 17. Resolution complete des captures

```
function resolveCaptures(state, player, sowingResult) {
  // Cas special du grenier : la derniere graine protegee est capturee seule.
  if (sowingResult.specialCapture > 0) {
    state.scores[player] += sowingResult.specialCapture;
    return {
      captured: sowingResult.specialCapture,
      type: "special-granary"
    };
  }

  const last = sowingResult.lastPosition;

  if (!canStartCapture(state, player, last)) {
    return {
      captured: 0,
      type: "none"
    };
  }

  const captureList = captureChainPositions(state, player, last);
  const captured = applyCaptureIfAllowed(state, player, captureList);

  return {
    captured,
    type: captured > 0 && captureList.length > 1 ? "chain" : "normal",
    cancelledBecauseStarvation: captured === 0 && captureList.length > 0
  };
}
```

Decision importante :

La capture speciale du grenier vaut toujours 1 graine.
Elle ne declenche pas de prise a la chaine.

18. 18. Coups interdits depuis la case d'attaque

La case d'attaque est :

Nord : N6
Sud : S0

Regles :

1 graine dans la case d'attaque : coup interdit en situation normale.
2 graines dans la case d'attaque : coup interdit en situation normale, sauf si le coup provoque une capture.

Pseudo-code :

```
function isAttackPitMove(player, pitIndex) {
  const attack = attackPit(player);
  return attack.player === player && attack.pitIndex === pitIndex;
}

function isForbiddenAttackMove(state, player, pitIndex) {
  if (!isAttackPitMove(player, pitIndex)) return false;

  const seeds = state.board[player][pitIndex];

  if (seeds === 1) {
    return true;
  }

  if (seeds === 2) {
    return !wouldMoveCapture(state, player, pitIndex);
  }

  return false;
}
```

Fonction `wouldMoveCapture` :

```
function wouldMoveCapture(state, player, pitIndex) {
  const simulated = cloneState(state);
  const sowing = sow(simulated, player, pitIndex);

  if (sowing.specialCapture > 0) {
    return true;
  }

  const last = sowing.lastPosition;
  return canStartCapture(simulated, player, last);
}
```

19. 19. Solidarite

La solidarite s'applique quand le camp adverse est vide avant le coup du joueur actif.

```
function opponentCampIsEmpty(state, player) {
  return sum(state.board[other(player)]) === 0;
}
```

Regle retenue :

Si le camp adverse est vide :

1. Le joueur doit donner au moins 7 graines a l'adversaire si possible.
2. Sinon, il doit donner le maximum de graines possible.
3. Si aucun coup ordinaire ne peut donner de graine, mais que le seul coup force est une case d'attaque avec 1 ou 2 graines, on applique le don force au score adverse.
4. Si aucun coup ne peut donner de graine, la partie s'arrete.

<!-- Rappel mi-fichier : Projet Songho | Auteur : NYUMEA PEHA DARYL G. | Matricule : 24H2571 |
Universite : UNIVERSITE DE YAOUNDE 1 | Niveau : LICENCE 2 | Pays : CAMEROUN -->

Calcul du nombre de graines donnees au camp adverse :

```
function countDeliveredToOpponent(state, player, pitIndex) {  
  const simulated = cloneState(state);  
  const before = sum(simulated.board[other(player)]);  
  sow(simulated, player, pitIndex);  
  const after = sum(simulated.board[other(player)]);  
  return after - before;  
}
```

Generation des coups de solidarite :

```

function getSolidarityMoves(state, player) {
  const candidates = ownNonEmptyMoves(state, player);

  const ordinary = candidates.filter((move) => {
    return !isForbiddenAttackMove(state, player, move.pitIndex);
  });

  const enriched = ordinary.map((move) => ({
    ...move,
    delivered: countDeliveredToOpponent(state, player, move.pitIndex)
  }));

  const atLeastSeven = enriched.filter((move) => move.delivered >= 7);
  if (atLeastSeven.length > 0) {
    return atLeastSeven;
  }

  const positive = enriched.filter((move) => move.delivered > 0);
  if (positive.length > 0) {
    const maxDelivered = Math.max(...positive.map((move) => move.delivered));
    return positive.filter((move) => move.delivered === maxDelivered);
  }

  const forcedDonation = candidates.filter((move) => {
    return isAttackPitMove(player, move.pitIndex)
      && [1, 2].includes(state.board[player][move.pitIndex]);
  });

  return forcedDonation.map((move) => ({
    ...move,
    forcedDonation: true
  }));
}

```

20. 20. Don force a l'adversaire

Cas :

Le joueur est contraint de jouer la case d'attaque avec 1 ou 2 graines.

Decision du projet :

Les graines sont retirees de la case source et ajoutees au score de l'adversaire.
 Il n'y a pas de semaille normale.
 Il n'y a pas de capture par le joueur actif.

Pseudo-code :

```
function applyForcedDonation(state, player, pitIndex) {
  const seeds = state.board[player][pitIndex];
  state.board[player][pitIndex] = 0;
  state.scores[other(player)] += seeds;

  return {
    type: "forced-donation",
    donated: seeds
  };
}
```

Cette decision doit etre signalee dans le rapport comme une convention de projet issue de la formulation des regles.

21. 21. Generation generale des coups legaux

```
function ownNonEmptyMoves(state, player) {
  const moves = [];

  for (let pitIndex = 0; pitIndex < 7; pitIndex++) {
    if (state.board[player][pitIndex] > 0) {
      moves.push({ player, pitIndex });
    }
  }

  return moves;
}

function getLegalMoves(state) {
  const player = state.currentPlayer;

  if (state.status !== "playing") {
    return [];
  }

  if (opponentCampIsEmpty(state, player)) {
    return getSolidarityMoves(state, player);
  }

  return ownNonEmptyMoves(state, player).filter((move) => {
    return !isForbiddenAttackMove(state, player, move.pitIndex);
  });
}
```

22. 22. Validation d'un coup

```

function validateMove(state, move) {
  if (state.status !== "playing") {
    return { ok: false, reason: "La partie est terminee." };
  }

  if (move.player !== state.currentPlayer) {
    return { ok: false, reason: "Ce n'est pas le tour de ce joueur." };
  }

  if (move.pitIndex < 0 || move.pitIndex > 6) {
    return { ok: false, reason: "Case inconnue." };
  }

  if (state.board[move.player][move.pitIndex] <= 0) {
    return { ok: false, reason: "La case est vide." };
  }

  const legalMoves = getLegalMoves(state);
  const legal = legalMoves.some((legalMove) => {
    return legalMove.player === move.player && legalMove.pitIndex === move.pitIndex;
  });

  if (!legal) {
    return { ok: false, reason: "Coup interdit par les regles." };
  }

  return { ok: true };
}

```

23.23. Application complete d'un coup

```

function applyMove(state, move) {
  const validation = validateMove(state, move);

  if (!validation.ok) {
    return {
      state,
      ok: false,
      error: validation.reason
    };
  }

  const legalMove = getLegalMoves(state).find((item) => {
    return item.player === move.player && item.pitIndex === move.pitIndex;
  });

  let actionResult;

  if (legalMove && legalMove.forcedDonation) {
    actionResult = applyForcedDonation(state, move.player, move.pitIndex);
  } else {
    const sowing = sow(state, move.player, move.pitIndex);
    const capture = resolveCaptures(state, move.player, sowing);
    actionResult = {
      type: "sow",
      sowing,
      capture
    };
  }

  state.moveNumber += 1;

  state.history.push({
    moveNumber: state.moveNumber,
    player: move.player,
    pitIndex: move.pitIndex,
    result: actionResult
  });

  resolveEndGameAfterMove(state);

  if (state.status === "playing") {
    state.currentPlayer = other(state.currentPlayer);
    resolveEndGameBeforeTurn(state);
  }

  assertTotalSeeds(state);

  return {
    state,
    ok: true,
    action: actionResult
  };
}

```

24. 24. Conditions de fin

La partie peut se terminer par :

Code	Condition
score_40	Un joueur atteint au moins 40 graines capturees.
low_board	Il reste moins de 10 graines sur le tablier.
solidarity_impossible	Un joueur ne peut pas etre nourri et ne peut pas jouer.
no_legal_move	Aucun coup legal n'existe.

25. 25. Attribution des graines restantes

Quand la fin exige de ramasser les graines du tablier :

```
function collectRemainingSeeds(state) {  
  state.scores.north += sum(state.board.north);  
  state.scores.south += sum(state.board.south);  
  state.board.north = [0, 0, 0, 0, 0, 0, 0, 0];  
  state.board.south = [0, 0, 0, 0, 0, 0, 0, 0];  
}
```

26. 26. Calcul du gagnant

```
function computeWinner(state) {  
  if (state.scores.north >= 40) return "north";  
  if (state.scores.south >= 40) return "south";  
  
  if (state.scores.north > state.scores.south) return "north";  
  if (state.scores.south > state.scores.north) return "south";  
  
  return "draw";  
}
```

Decision :

Le seuil officiel de victoire est 40.
Pour les fins par ramassage, si personne n'atteint 40 mais qu'un score est superieur, on peut afficher le meilleur score.
Si les deux scores sont egaux, afficher match nul.

Pour rester strict avec Club Awale :

Une vraie victoire automatique est `score >= 40`.
Une fin sans `score >= 40` est un nul officiel si l'enseignant veut appliquer strictement "aucun joueur au-dessus de 39".

Dans le code, prévoir une option :

```
const ENDING_POLICY = "strict_40_or_draw"; // ou "highest_score_wins"
```

Pour le projet scolaire, utiliser par défaut :

```
strict_40_or_draw
```

27. 27. Fin apres un coup

```
function resolveEndGameAfterMove(state) {  
  if (state.scores.north >= 40 || state.scores.south >= 40) {  
    state.status = "ended";  
    state.reason = "score_40";  
    state.winner = computeWinner(state);  
    return;  
  }  
  
  if (boardSeeds(state) < 10) {  
    collectRemainingSeeds(state);  
    state.status = "ended";  
    state.reason = "low_board";  
    state.winner = computeWinnerStrict40OrDraw(state);  
    return;  
  }  
}
```

Version stricte du gagnant :

```
function computeWinnerStrict40OrDraw(state) {  
  if (state.scores.north >= 40) return "north";  
  if (state.scores.south >= 40) return "south";  
  return "draw";  
}
```

28. 28. Fin avant le tour suivant

Avant de laisser le joueur suivant jouer, verifier s'il existe au moins un coup legal.

```
function resolveEndGameBeforeTurn(state) {
  const legalMoves = getLegalMoves(state);

  if (legalMoves.length > 0) {
    return;
  }

  collectRemainingSeeds(state);

  state.status = "ended";
  state.reason = "solidarity_impossible";
  state.winner = computeWinnerStrict40OrDraw(state);
}
```

29. 29. Verification de l'invariant

```
function assertTotalSeeds(state) {
  const total = totalSeeds(state);

  if (total !== 70) {
    throw new Error(`Invariant casse : ${total} graines au lieu de 70.`);
  }
}
```

Cette verification est tres importante dans la version Ajax, car elle detecte vite :

- une double execution d'un coup ;
- une mauvaise capture ;
- une mauvaise synchronisation ;
- une mauvaise gestion du grenier.

30. 30. Ordre complet d'un tour

1. Lire l'etat courant.
2. Verifier que la partie est en cours.
3. Verifier que le joueur demandeur est le joueur courant.
4. Verifier que la case appartient au joueur.
5. Verifier que la case n'est pas vide.
6. Generer les coups legaux.
7. Verifier que le coup choisi est legal.
8. Si c'est un don force, appliquer le don.
9. Sinon, semer les graines.
10. Resoudre la capture speciale du grenier si elle existe.
11. Sinon, resoudre la capture normale et la prise a la chaine.
12. Annuler la capture si elle vide le camp adverse.
13. Ajouter le coup dans l'historique.
14. Verifier score ≥ 40 .
15. Verifier tablier < 10 graines.
16. Changer le joueur courant.
17. Verifier que le joueur suivant a un coup legal.
18. Verifier l'invariant des 70 graines.
19. Retourner l'etat mis a jour.

31. 31. Cas de test obligatoires

31.1. 31.1 Etat initial

```
north = [5,5,5,5,5,5,5]
south = [5,5,5,5,5,5,5]
scores = 0 / 0
total = 70
```

Attendu :

- partie en cours ;
- total 70 ;
- joueur courant defini ;
- 7 cases par joueur.

31.2. 31.2 Semaille normale Sud

Etat :

```
Sud S2 = 5
```

Coup :

```
Sud joue S2
```

Chemin :

S1, S0, N0, N1, N2

Attendu :

- S2 devient 0 ;
- les 5 cases du chemin gagnent 1 graine.

31.3. 31.3 Semaille normale Nord

Etat :

Nord N4 = 5

Coup :

Nord joue N4

Chemin :

N5, N6, S6, S5, S4

Attendu :

- N4 devient 0 ;
- les 5 cases du chemin gagnent 1 graine.

31.4. 31.4 Grenier avec 14 graines

Etat :

Sud S3 = 14

Attendu :

- S3 devient 0 ;
- les 13 autres cases gagnent 1 graine ;
- la 14e graine est capturee par Sud ;
- score Sud augmente de 1 ;
- aucune capture a la chaine n'est declenchee.

31.5. 31.5 Grenier avec 15 graines

Etat :

Sud S3 = 15

Attendu :

- les 13 autres cases gagnent 1 graine ;
- la 14e graine va en N0 ;
- la 15e graine va en N1 ;
- pas de capture speciale ;
- capture normale possible seulement si N1 contient 2, 3 ou 4 apres semaille.

31.6. 31.6 Capture normale

Etat apres semaille :

Derniere graine chez l'adversaire.
Case finale = 3 graines.
Case finale != premiere case adverse.

Attendu :

- la case est capturee ;
- le score du joueur actif augmente de 3 ;
- la case passe a 0, sauf si cela vide tout le camp adverse.

31.7. 31.7 Capture interdite dans la premiere case adverse

Etat apres semaille :

Derniere graine dans la premiere case adverse.
Cette case contient 3 graines.

Attendu :

- pas de capture normale ;
- pas de prise a la chaine.

31.8. 31.8 Prise a la chaine incluant la premiere case adverse

Etat apres semaille pour Sud :

N0 = 3
N1 = 4
N2 = 2
Derniere graine en N2

Attendu :

- capture N2 ;
- capture N1 ;
- capture N0 ;
- la premiere case adverse est capturee car elle est atteinte par la chaine.

31.9. 31.9 Capture qui affame l'adversaire

Etat :

La capture prevue retire toutes les graines du camp adverse.

Attendu :

- aucune capture n'est appliquee ;
- la semaille reste appliquee ;
- le score ne change pas pour cette capture.

31.10. 31.10 Coup interdit depuis la case d'attaque avec 1 graine

Etat :

Sud S0 = 1
Camp Nord non vide

Attendu :

- coup interdit.

31.11. 31.11 Coup interdit depuis la case d'attaque avec 2 graines sans capture

Etat :

Sud S0 = 2
Le coup ne provoque aucune capture.

Attendu :

- coup interdit.

31.12. 31.12 Coup autorise depuis la case d'attaque avec 2 graines si capture

Etat :

Sud S0 = 2
Le coup finit en N1 avec 2, 3 ou 4 graines.

Attendu :

- coup autorise ;
- capture appliquee si elle n'affame pas Nord.

31.13. 31.13 Solidarite avec au moins 7 graines donnees

Etat :

Camp Nord vide.
Sud peut jouer un coup qui donne 7 graines ou plus a Nord.

Attendu :

- seuls les coups qui donnent au moins 7 graines sont proposes.

31.14. 31.14 Solidarite avec maximum possible

Etat :

Camp Nord vide.
Aucun coup Sud ne peut donner 7 graines.
Sud peut donner au maximum 4 graines.

Attendu :

- seuls les coups donnant 4 graines sont proposes.

31.15. 31.15 Solidarite impossible

Etat :

Camp Nord vide.
Sud ne peut donner aucune graine a Nord.

Attendu :

- partie terminee ;
- graines restantes attribuees aux proprietaires ;
- gagnant determine selon la politique stricte du score 40.

31.16. 31.16 Fin par moins de 10 graines

Etat apres coup :

```
Total sur tablier = 9
```

Attendu :

- ramassage des graines restantes ;
- tablier vide ;
- partie terminee.

31.17. 31.17 Fin par score 40

Etat apres capture :

```
Score Sud = 40
```

Attendu :

- partie terminee ;
- Sud gagnant.

32. 32. Organisation conseillee du moteur

Fichiers communs aux deux versions :

```
js/core/constants.js
js/core/coordinates.js
js/core/state.js
js/core/sowing.js
js/core/capture.js
js/core/legal-moves.js
js/core/end-game.js
js/core/engine.js
js/core/tests.js
```

Responsabilites :

Fichier	Role
<code>constants.js</code>	Constantes du jeu.
<code>coordinates.js</code>	Cycle, chemins, conversions d'affichage.
<code>state.js</code>	Creation, clonage, invariant.
<code>sowing.js</code>	Semille normale et grenier.
<code>capture.js</code>	Capture normale, chaine, anti-affamement.
<code>legal-moves.js</code>	Coups autorises, interdits, solidarite.
<code>end-game.js</code>	Conditions de fin, gagnant, ramassage.
<code>engine.js</code>	Fonction publique <code>applyMove</code> .
<code>tests.js</code>	Tests unitaires simples.

33.33. Regle d'or de programmation

Le moteur du jeu ne doit jamais lire le DOM et ne doit jamais faire d'Ajax.

Il recoit seulement :

```
etat + coup
```

Et retourne :

```
nouvel etat + resultat du coup
```

Cette separation permet :

- de reutiliser le moteur dans la version locale ;
- de reutiliser le moteur cote serveur dans la version distante si le serveur est en JavaScript ;
- de tester les regles sans interface ;
- d'eviter les divergences entre les deux versions.